成都理工大学中英合作办学
CDUT Sino-British Collaborative Education

OXFORD BROOKES UNIVERSITY

UNDERGRADUATE PROJECT REPORT

Project Title	ProsePal:An Intelligent Writing Assistant for Web Novel Authors
Student ID	202218020401
Supervisor Name	Clivia Li
Student Name	Hanna
Student Major	Software Engineering

6th May 2026

Date Submitted

Declaration

Chengdu University of Technology Oxford Brookes College

Chengdu University of Technology

BSc (Single Honours) Degree Project

Programme Name:ProsePal

Module No.:CHC6096

Surname:Lin

First Name: Junxi

Project Title: ProsePal:An Intelligent Writing Assistant for Web Novel Authors

Student No.: 202218020401

Supervisor: Clivia

2ND Supervisor (if applicable): Not Applicable

A report submitted as part of the requirements for the degree of BSc (Hons) in Software Engineering

At

Chengdu University of Technology Oxford Brookes College

Date Submitted：6th May 2026

Student Conduct Regulations

Please ensure you are familiar with the regulations regarding Academic Integrity. The University takes this issue very seriously, and students have been expelled or had their degrees withheld for cheating in assessments. Students experiencing difficulties with their work must seek help from their tutors rather than being tempted to use unfair means to gain marks. Students should not risk losing their degree and undermining all the work they have done towards it. You are expected to have familiarised yourself with these regulations. https://www.brookes.ac.uk/regulations/current/appeals-complaints-and-conduct/c1-1/

Guidance on the correct use of references can be found on www.brookes.ac.uk/services/library, and also in a handout in the library. The full regulations may be accessed online at https://www.brookes.ac.uk/students/sirt/student-conduct/. If you do not understand what any of these terms mean, you should ask your Project Supervisor to clarify them for you.

I declare that I have read and understood Regulations C1.1.4 of the Regulations governing Academic Misconduct and that the work I submit is fully in accordance with them.

Signature: Hanna Junxi Lin Date: 6th May 2026

Regulations Governing the Deposit and Use of Oxford Brookes University Modular Programme Projects and Dissertations

Copies of projects/dissertations, submitted in fulfilment of Modular Programme requirements and achieving marks of 60% or above, shall normally be kept by the Oxford Brookes University Library.

I agree that this dissertation may be available for reading and photocopying in accordance with the Regulations governing the use of the Oxford Brookes University Library.

Signature: Hanna Junxi Lin Date: 6th May 2026

Acknowledgement

First and foremost, I would like to express my deepest gratitude to my supervisor, Clivia Li. Throughout the entire process of my thesis research and writing, from the initial topic selection, research framework design to the revision and improvement of the full text, She has provided me with meticulous guidance, valuable professional advice and unwavering support. Her rigorous academic attitude, profound professional knowledge and diligent research spirit have set an excellent example for me, which will benefit me a lot in my future academic research and work.

I would also like to thank all the teachers who have taught me during my study at Chengdu University of Technology. Their systematic teaching and patient instruction have laid a solid theoretical foundation for my thesis research, broadened my academic vision, and enabled me to continuously improve my professional ability and research thinking.

My sincere thanks go to my classmates and friends. During the period of thesis writing, we discussed academic problems together, shared research experiences and encouraged each other. Their company and help made the arduous research process full of warmth and strength, and also allowed me to overcome many difficulties in the research.

I am particularly grateful to my family for their selfless love, silent support and full understanding. They are my strongest backing, always encouraging me to pursue my academic goals, and their care and companionship have always been the driving force for me to keep moving forward.

Finally, I would like to extend my heartfelt thanks to all the individuals and institutions that have provided help and support for this research. Although the thesis is completed, the road of academic exploration is long. I will take this as a starting point, continue to study hard and forge ahead in the future study and work.

Abstract

The rapid expansion of web literature has highlighted the inadequacies of general-purpose word processors, which often cause cognitive overload and fragmented workflows for authors managing long-form, complex narratives. This project aims to design and implement "ProsePal," an intelligent, full-stack writing assistant tailored specifically to enhance the productivity, narrative consistency, and collaborative experience of web novel creators.

Developed using an Agile methodology with Vue.js, Node.js, and MySQL, the platform integrates the Llama-3 Large Language Model (LLM) API to provide advanced NLP capabilities. A significant technical achievement of this project is the implementation of a dual-layer Map-Reduce caching architecture. By utilizing database-level summaries and MD5-hashed file caching, the system effectively resolves LLM context-window limitations and API latency during large-scale character timeline generation. Furthermore, the system incorporates Retrieval-Augmented Generation (RAG) to ensure plot consistency via a dynamic "World Bible," alongside a centralized dashboard featuring a context-bound "Pending Revisions" loop for streamlined asynchronous editorial collaboration.

Comprehensive unit and integration testing confirmed the stability and high performance of the deployed architecture. Ultimately, ProsePal successfully bridges the gap between advanced AI capabilities and practical literary creation, delivering a stable, distraction-free environment that mitigates cognitive burden and modernizes the manuscript development lifecycle.

Keywords: **Intelligent Writing Assistant, Web Novel Creation, Large Language Models, Retrieval-Augmented Generation (RAG), Map-Reduce Caching**

Abbreviations

ACM	Association for Computing Machinery
AI	Artificial Intelligence
API	Application Programming Interface
BCS	British Computer Society
BSc	Bachelor of Science
CPU	Central Processing Unit
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
GDPR	General Data Protection Regulation
GUI	Graphical User Interface
HTML	HyperText Markup Language
HTTP / HTTPS	HyperText Transfer Protocol / Hypertext Transfer Protocol Secure
I/O	Input/Output IPIntellectual Property
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MD5	Message-Digest Algorithm 5
MVP	Minimum Viable Product
NLP	Natural Language Processing
OT	Operational Transformation
RAG	Retrieval-Augmented Generation
RAM	Random Access Memory
SPA	Single Page Application
SQL	Structured Query Language
TDD	Test-Driven Development
UAT	User Acceptance Testing
UI / UX	User Interface / User Experience
WCAG	Web Content Accessibility Guidelines XSSCross-Site Scripting

Glossary

Term	Definitions
ProsePal	The proprietary intelligent writing assistant platform developed in this project, designed specifically to address the complex needs of web novel authors.
Intelligent Writing Assistant	A software ecosystem that utilizes artificial intelligence to aid users in the writing process, offering features beyond basic word processing, such as narrative analysis and logic checking.
Web Novel	A serialized work of fiction published primarily on the internet, typically characterized by extreme length, massive character rosters, and frequent chapter updates
Large Language Model (LLM)	An advanced artificial intelligence system trained on massive text datasets, capable of understanding and generating human-like language, utilized in this project for deep narrative analysis.
Retrieval-Augmented Generation (RAG)	An AI architecture that grounds the outputs of an LLM by first retrieving relevant facts from an external knowledge base before generating a response, effectively preventing AI hallucinations.
Map-Reduce Caching	A specialized algorithmic and storage strategy implemented in ProsePal. It divides long texts for individual summarization (Map), combines the results (Reduce), and caches them using MD5 hashes to prevent redundant API computation.
Collaborative Editing	The asynchronous workflow where multiple users (authors and editors) interact with the same manuscript via context-bound annotations and a centralized "Pending Revisions" loop.
World Bible (Lore Entry)	A structured database feature within ProsePal used to meticulously track character profiles, settings, and fictional rules, acting as the foundational grounding truth for both the author and the RAG system.
Content Fingerprinting	A caching technique used in the project's backend where unique MD5 hashes are generated based on manuscript content to instantly verify if an AI timeline summary already exists.

Table of Contents

Chapter 1. Introduction

1.1 Background of Study

With the rise of digital media, web literature has become a lifeful and significant component of the global cultural industry, attracting hundreds of millions of readers and creators [1]. According to recent academic studies, by mid-2025, the number of Chinese online literature users had reached 513 million, accounting for 45.7% of the total internet population [2]. After more than two decades of rapid development, Chinese web novels have accelerated their global reach, expanding from East Asia to North America and gaining a broad international market space [3]. Furthermore, by the end of 2022, the domestic online literature market size had already reached 38.93 billion CNY, underscoring its immense economic capacity and its vital role as a major resource for intellectual property (IP) transformation [4].However, despite this creative boom, many authors still rely on universal office software such as Microsoft Word, Google Docs for their work. While powerful, these tools were designed for general document processing and fall short of meeting the complex demands of long-form narrative works, especially web novels, which require functionalities like non-linear chapter management, visualization of character relationships, and check the coherence of the plot [5].

Creators often face too many challenges,such as including frequently switching between multiple applications to consult research data, manage ideas, and organize outlines, which significantly fragments their creative focus and reduces efficiency. Furthermore, the manuscripts revision process often relies on subjective communication between authors and editors,lacking of tools for data analysis to assist in revision.The advancement of modern computational linguistics and human-computer interaction offers a potential solution to these pain points [6]. Therefore, develop a writing assistant system specifically designed for web novel author—integrating intelligent analysis and efficient collaboration—is of significant practical importance and applicational value.

1.2 Project Aim

The primary aim of this project is to design and implement an integrated software called "ProsePal." This system aims to significantly enhance the creative

output, quality of works, and collaborative experience of online novel authors by providing a platform that integrates creative conception, writing, analysis, and collaboration.

1.3 Project Objectives

To achieve the project aim, the following types of work will be performed:

- Requirements Gathering and Analysis Work: To elicit authentic user needs and pain points through in-depth industry interviews and user personas, alongside a comparative analysis of existing writing tools to define functional baselines and identify key market gaps.
- Research and Analysis Work: To conduct a comparative analysis of existing writing tools to define functional baselines and identify key market gaps.
- System Design Work: To perform system and software architecture design, including the creation of comprehensive database schemas and detailed API specifications.
- User Interface Design Work: To design high-fidelity UI mockups and interactive wireframes, focusing on minimalism and cognitive load reduction to ensure an intuitive user experience for both authors and editors.
- Implementation and Development Work: To program the front-end and back-end modules of the application, including the core text editor, document management system, and intelligent analysis etc.
- Integration Work: To integrate the core application with third-party services and libraries, specifically for NLP functionalities and real-time collaboration features.
- Testing and Quality Assurance Work: To execute a comprehensive testing strategy, including the development of unit tests, the execution of integration tests, and the coordination of user acceptance testing to ensure software stability and reliability.

1.4 Project Overview

This project aims to develop an intelligent writing assistant software specifically designed for web novel creators. It combines traditional text editing function with advanced data analysis, project management, and online collaboration tools to solve the efficiency bottlenecks and the difficulties of creat that authors currently face in their workflow.

1.4.1 **Scope**

The software provides a highly focused, AI-enhanced creative environment through the following core functionalities:

Core Text Editing Workspace:

- Rich-Text Editing Interface: As the foundational tool for authors, the platform provides a robust and intuitive rich-text editor. It supports standard typographic formatting (e.g., font styles, sizing, paragraph alignment) specifically tailored for drafting and formatting long-form manuscripts.
- Distraction-Free Environment: The writing interface adopts a minimalist design philosophy, allowing authors to collapse secondary toolbars and sidebars (such as the AI Assistant or World Bible) to minimize visual clutter and maintain deep creative focus.

Intelligent Document Management:

- Centralized Dashboard Analytics: A comprehensive dashboard provides real-time tracking of author productivity, displaying total word counts, active projects, and instant notifications for pending editorial revisions.
- Dynamic Document Structuring: Authors can seamlessly manage manuscript chapters through a drag-and-drop interface, allowing for visual restructuring of the plot.
- Automated State Management (Auto-Save): The system features a robust, debounced auto-save mechanism that quietly syncs manuscript modifications to the database in real-time, preventing data loss without disrupting the creative flow.

Dynamic World Building System:

- Structured Lore Management: Instead of a simple note-taking tool, the system provides a "World Entry" database where authors can create detailed profiles for characters, locations, and items (including image uploads and rich-text rules).
- Contextual Integration: These world entries act as the foundational grounding truth for the narrative, keeping the author aligned with their own established universe during the writing process.

Advanced NLP & AI-Powered Assistance:

- Character Timeline Generation: Utilizing a sophisticated dual-layer caching architecture (Database-level summary caching and MD5-hashed file caching), the AI can instantly extract and generate a chronological timeline of specific

character arcs across the entire manuscript without redundant full-text traversals.

- Context-Aware Text Polishing: An integrated AI editor allows authors to apply targeted literary enhancements, grammar fixes, or stylistic adjustments to selected text blocks.
- World View Consultation (RAG Implementation): Authors can query an intelligent co-author assistant. By utilizing Retrieval-Augmented Generation (RAG) principles, the AI analyzes the full manuscript alongside selected "World Entries" to provide plot suggestions and answer lore questions while strictly adhering to established facts.

Streamlined Editorial Collaboration:

- Targeted Annotations & Comments: Editors and co-authors can select specific text segments within the manuscript to leave precise, context-bound feedback or revision requests.
- "Pending Revisions" Notification Loop: The system automatically aggregates all unresolved editorial comments and pushes them to the author's dashboard as "Pending Revisions," allowing the author to quickly navigate to the exact chapter and line that requires attention.

1.4.2 Audience

The findings and deliverables of this project will primarily benefit the following groups:

- Web Novel Authors: As the core users, they will directly benefit from a more professional and intelligent creative environment, allowing them to focus more on the art of storytelling.
- Content Editors and Planners: They can leverage the system's collaborative and analytical features to guide authors more effectively, assess manuscript quality, and gain a high-level understanding of a work's structure and potential.

1.5 Structure of the Project Report

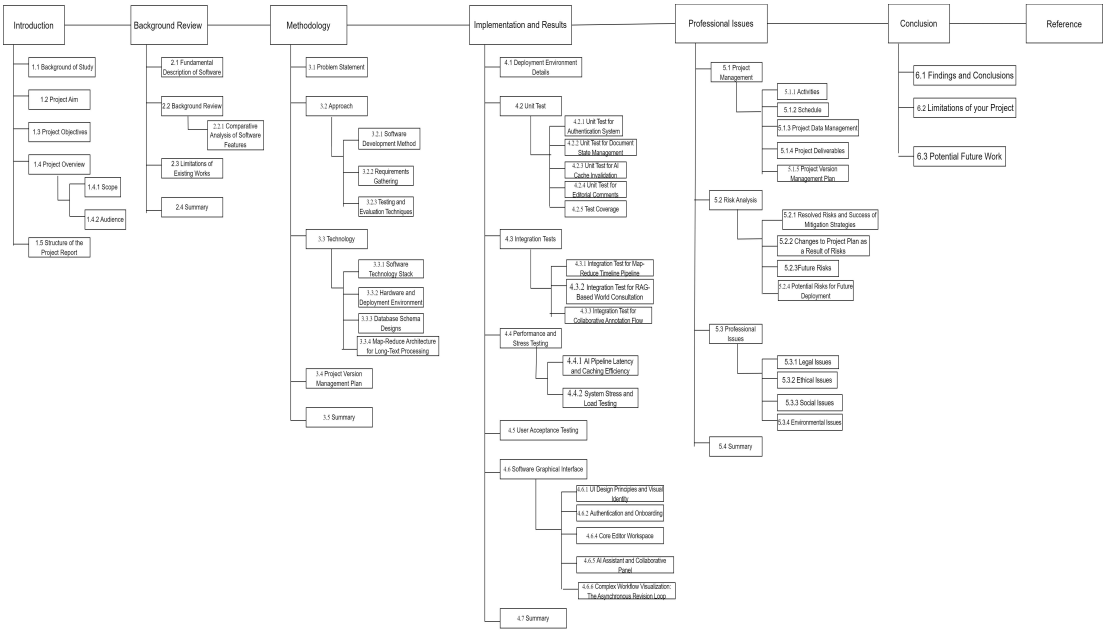


Figure 1-1. Thesis structure flowchart

Chapter 2. Background Review

2.1 Fundamental Description of Software

ProsePal is an intelligent, full-stack novel creation platform designed to significantly enhance authorial productivity and narrative consistency by integrating advanced computational linguistics with a streamlined creative workflow. Moving beyond the capabilities of general-purpose word processors, the primary purpose of ProsePal is to provide a specialized, distraction-free ecosystem that mitigates the cognitive load of authors managing long-form, complex narratives. The software achieves this by bridging the gap between traditional manuscript drafting and modern AI-driven data management.

The core functionality of the platform is structured around four primary pillars:Intelligent

- Document and Project Management: The system features a centralized dashboard that provides real-time tracking of productivity metrics, such as total word counts and project status, combined with a dynamic chapter management sidebar and a robust, debounced auto-save mechanism to ensure data integrity without disrupting creative flow.
- AI-Powered Narrative Intelligence: Utilizing a sophisticated dual-layer Map-Reduce caching architecture (Database-level summaries and MD5-hashed JSON files), the platform can instantly extract and generate chronological character timelines across hundred-thousand-word manuscripts, effectively overcoming the token limits of standard Large Language Models (LLMs).
- Context-Aware World Building (RAG): Through the implementation of Retrieval-Augmented Generation (RAG), ProsePal provides a "World Bible" feature. This allows authors to create structured lore entries for characters and settings which act as the grounding truth for an AI assistant, ensuring that plot suggestions and consistency checks strictly adhere to the author's established universe.
- Streamlined Asynchronous Collaboration: The system facilitates a precise feedback loop between authors and editors through context-bound annotations. All unresolved comments are automatically aggregated into a "Pending Revisions" notification loop on the dashboard, allowing users to

navigate directly to specific chapter segments that require attention.

2.2 Background Review

2.2.1 Comparative Analysis of Software Features

This section provides a comparative analysis of existing writing software relevant to this project. These tools can be categorized into three types: general document tools, professional writing software, and existing web novel writing tools. The analysis will focus on their features regarding document management, collaboration capabilities, and intelligent analysis.

Table 2-1. Format of Table

Authoring Tools	Document Management (Version Control, Outline)	Collaboration Features	Intelligent Analysis (Character Mapping, Plot Curve)	Proofreading check
Microsoft Word[7]	Yes	Yes	No	Yes
Google docs[8]	Yes	Yes	No	Yes
Scrivener[9]	Yes	Yes	No	No
Ulysses[10]	Yes	Yes	No	No
Writer's assistant	Yes	Yes	No	Yes
Orange melon writing	Yes	No	No	Yes
Our Proposed System	Yes	Yes	Yes	Yes

2.3 Limitations of Existing Works

The analysis above reveals a clear market gap in existing tools:

- General Document Tools lack professional features designed for long-form writing, such as complex chapter management and character tracking, leading to a fragmented creative process.
- Professional Writing Software, while excellent for structural management, generally has weak or non-existent collaboration features, failing to meet the close communication needs between authors and editors.
- Existing Web Novel Tools often have basic functionalities and weak analytical capabilities, failing to fully leverage data analysis techniques to assist creative

decision-making.

In summary, the current market lacks an integrated solution that seamlessly combines a fluid writing experience, deep narrative intelligence, and efficient online collaboration.

2.4 Summary

This chapter provided a rigorous background review, transitioning from a general software description to a detailed comparative analysis of the current market. By evaluating ProsePal's three-layer architecture (Drafting, Narrative Engine, and Collaboration) against traditional tools like Microsoft Word and Google Docs, a significant functional gap was identified in handling complex character mapping and plot logic for large-scale manuscripts. The review established that for a 38.9 billion CNY market, existing general-purpose processors are insufficient. These findings justify the necessity of the proposed system’s specialized features, particularly its ability to maintain narrative consistency through integrated AI analysis.

Chapter 3. Methodology

This chapter details the methodology adopted to solve the aforementioned problems, including a clear problem statement, software development approach, technology selection, and version management plan.

3.1 Problem Statement

Web novel authors face three core problems in their creative process:

- Cognitive Overload and Disconnected Lore Management: Authors frequently struggle to maintain consistency across extensive fictional universes because world-building reference materials (characters, settings, rules) and the actual manuscript exist in fragmented silos. Constantly context-switching to reference or verify lore disrupts the creative flow and severely increases the cognitive load of knowledge workers [11].
- Difficulty in Tracking Macro-Narratives Across Massive Contexts: Long-form novels involve hundreds of thousands of words, making it nearly impossible for human authors to accurately track specific character arcs. Furthermore, attempting to use standard AI tools for this task often fails due to strict LLM token limits and the well-documented "lost in the middle" phenomenon [12], leading to inevitable plot holes and logical inconsistencies.
- Unstructured and Disconnected Editorial Workflows: The manuscript revision process between authors and editors often relies on external communication channels or generic file exchanges. Research shows that when feedback is not strictly bound to the exact narrative context, asynchronous collaborative writing suffers from severe communication overhead. Authors lack a centralized dashboard to track pending revisions, resulting in long, chaotic, and inefficient editing cycles.

3.2 Approach

3.2.1 Software Development Method

Agile development is an iterative and incremental methodology that prioritizes flexibility, continuous testing, and the rapid delivery of functional software components by breaking the project lifecycle into manageable, time-boxed

"Sprints."This approach was selected as the optimal framework for the ProsePal project due to the inherent unpredictability of integrating Large Language Models (LLMs), where factors such as prompt effectiveness and API latency required a highly adaptable environment for rapid prototyping.By adopting agile development, it is possible to continuously optimize complex functions based on real-time test results, such as RAG context injection and database performance.

The implementation of the agile development method in this project followed a structured process based on Scrum:

- Backlog & Planning Phase: A prioritized product backlog was established to distinguish 'must-have' core features from 'nice-to-have' enhancements, allowing the development to be systematically divided into many iterative versions.
- Iterative Development & Testing: During each sprint, frontend components in Vue.js and backend APIs in Node.js were developed concurrently, with incremental code subjected to rigorous unit testing using Jest to ensure stability before full integration.
- Sprint Review & Technical Adaptation: At the conclusion of each cycle, the working software was evaluated to identify performance bottlenecks. A key success of this stage was identifying high API latency during the initial AI integration, which led to the immediate Agile-driven decision to design and implement the MD5-hashed dual-layer caching system in the subsequent sprint.

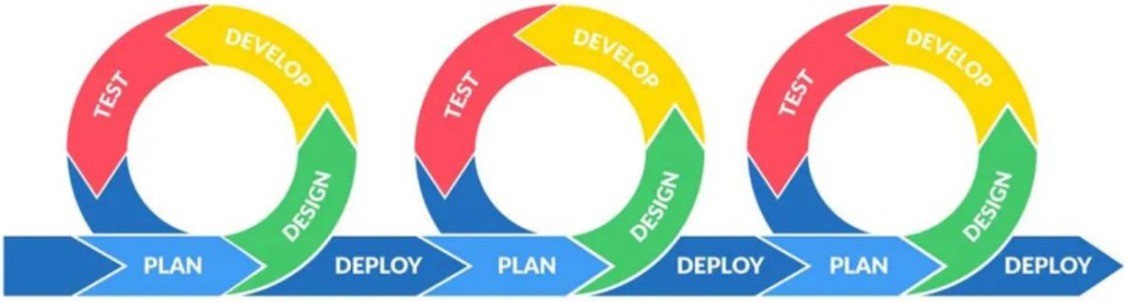


Figure 3-1. Image of Agile

3.2.2 Requirements Gathering

A multi-faceted approach will be used for requirements elicitation:

- Indirect Interviews: To deeply understand the actual workflows and pain points of the target audience, a structured interview was conducted with a professional

web novel editor (The full interview transcript is provided in Appendix A). The interview revealed that authors critically need robust auto-save functionalities, version comparison, and hierarchical outlining. More importantly, it highlighted that the most frequent editorial feedback revolves around maintaining character consistency, logical continuity, and plot pacing. Current reliance on generic tools like Google Docs or offline MS Word fragments the workflow, especially during manuscript revision. These direct industry insights justified the development of ProsePal's dedicated "World Bible", version control, and context-bound "Pending Revisions" features.

- Competitive Analysis: Systematically analyzing existing writing software to establish functional baselines and identify opportunities for innovation.
- User Personas: Creating typical user personas of web novel authors to ensure design decisions remain user-centric.

3.2.3 Testing and Evaluation Techniques

- Structural and Unit Testing: Core backend modules, particularly the aiController and documentController, will undergo rigorous unit testing to verify individual functions in isolation. White-box testing techniques will be applied to the complex algorithmic components.
- System Integration Testing: This phase focuses on verifying the seamless data exchange between the Vue.js frontend, the Node.js backend services, and the MySQL database. Crucially, integration tests will heavily target the external Large Language Model (LLM) API connections. This includes testing asynchronous timeout handling, proxy network stability, and the accurate parsing of Map-Reduce JSON payloads between the application and the AI service.
- User Acceptance Testing (UAT): In the final stages, qualitative feedback will be gathered by inviting target users to simulate real-world workflows. Testers will assume specific roles "Authors" and "Editors" to evaluate the role-based access control, the accuracy of the Retrieval-Augmented Generation (RAG) world-building outputs, and the practical usability of the real-time "Pending Revisions" dashboard.

A detailed execution of these testing strategies, along with the specific results and interface evaluations, will be thoroughly discussed in Chapter 4. Implementation and Results .

3.3 Technology

3.3.1 Software Technology Stack

Table 3-1. Table of software configuration

Category	Tools	Purpose
Frontend	Vue.js+TypeScript	Building a responsive, interactive user interface and managing component states (e.g., Dashboard, Editor).
Backend	Node.js[13]+ Express	Handling RESTful API requests, business logic, and dual-layer caching orchestration.
Database	MySQL	Storing structured data including projects, documents, editorial comments, and AI summary caches.
AI Services	LLM API (e.g., Llama-3 / OpenAI SDK)	Executing Map-Reduce timeline generation, RAG-based world building consultation, and contextual text polishing.
Data Caching.	Node.js fs + Crypto (MD5)	Generating content fingerprints and storing compiled JSON timelines for zero-latency retrieval.
Testing Tools	Jest[14]	Unit test, API test

3.3.2 Hardware and Deployment Environment

Table 3-2. Table of hardware enviroment

Enviroment	Tools	Purpose
Development Enviroment	Developer PC	Used for local code development, frontend UI rendering, and running local unit tests.
Production Environment	Cloud Server	Used to deploy the Node.js backend, host the MySQL database, and process

live API requests in the production environment.

3.3.3 **Database Schema Designs**

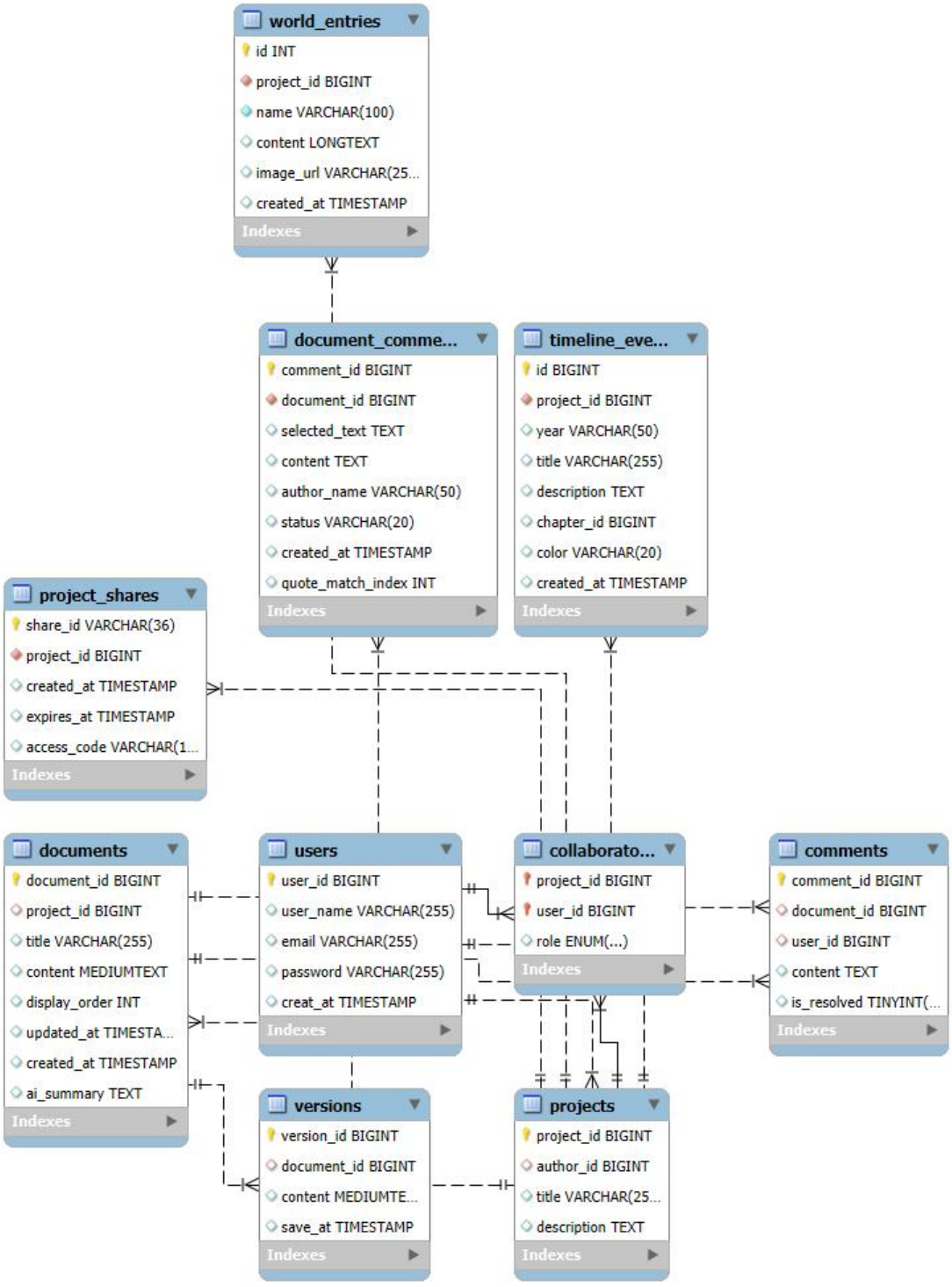


Figure 3-2. Image of Database Schema Design

Figure 3-2 illustrates the Entity-Relationship (ER) diagram of the ProsePal platform, designed to efficiently manage complex hierarchical data, facilitate real-time collaboration, and optimize operations for the integrated Large Language Model (LLM). The database is strictly normalized to ensure data integrity and is structured around four core functional domains.

1.Core Structure and Content Management

At the heart of the system is the projects table, which serves as the root container for a novel. A single project has a one-to-many relationship with the documents table, representing individual chapters.

- Design Rationale: By decoupling a massive manuscript into discrete documents with a display_order integer, the system avoids loading hundreds of thousands of words into the browser's memory simultaneously.

2. AI Optimization and RAG Integration

The database is uniquely tailored to support advanced AI features efficiently. The documents table includes an ai_summary field. Additionally, the world_entries table stores character and setting lore, while timeline_events stores generated chronologies.

- Design Rationale: The ai_summary field is the cornerstone of the dual-layer caching strategy. By persistently storing chapter-level AI reductions in MySQL, the system drastically cuts down external LLM API costs and latency. Furthermore, isolating world_entries into its own table allows the backend to dynamically query and inject specific lore into the LLM context window (RAG), preventing AI hallucinations without exceeding token limits.

3. Precision Collaboration and Feedback

The system implements a robust collaboration layer through the collaborators, project_shares, and document_comments tables.

- Design Rationale: Unlike generic text editors, ProsePal's document_comments table is designed for high-precision, context-bound feedback. By storing the selected_text and a quote_match_index, the database ensures that editorial revisions remain perfectly anchored to the exact sentence intended, resolving the severe communication overhead typically found in asynchronous editing. The project_shares table further enhances security by providing temporary,

token-based (access_code) read-only access for external reviewers.

4. Data Security and Version Control

To mitigate the risk of data loss, the database employs a versions table linked to each document.

- Design Rationale: Every time the auto-save mechanism is triggered or a significant edit is made, a snapshot is appended to the versions table. This append-only approach guarantees complete traceability and allows authors to roll back to previous manuscript states, fulfilling the project's data safety requirements.

3.3.4 Map-Reduce Architecture for Long-Text Processing

During the initial development and integration of the large language model (LLM), the system encountered significant technical challenges when dealing with long documents. Specifically, when attempting to analyze a complete 100,000-word chapter of a web novel, API timeout errors and token window overflow issues often occurred. Moreover, real-time repetitive processing of the same text led to excessively high API costs and severe delays, making functions such as "character timeline generation" almost unusable for professional-length works.

To overcome these obstacles, a Map-Reduce algorithmic architecture was designed and implemented as a specialized solution. This approach shifts the processing philosophy from "full-text submission" to "distributed summarization and synthesis," effectively bypassing hardware and API limitations. The systematic execution of this process is visualized in the following flowchart:

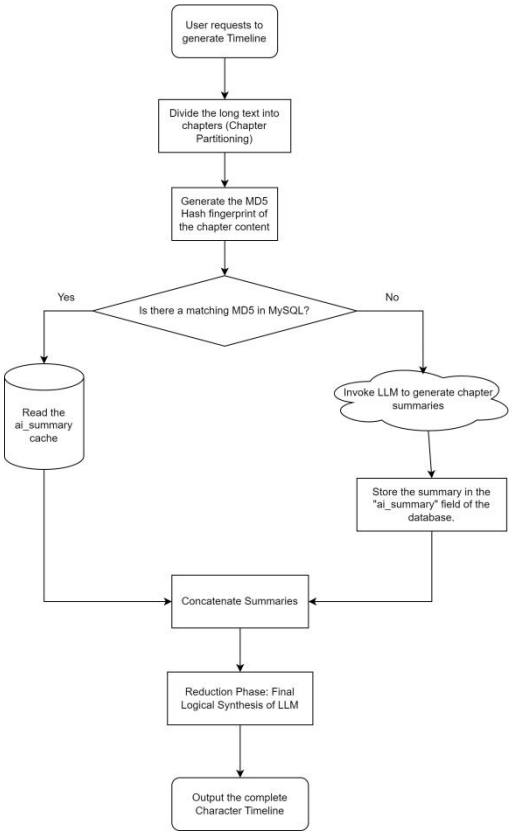


Figure 3-3. Logic flow of the Map-Reduce and MD5 caching architecture

- Phase 1: Map (The Chunking & Caching Layer): The system first partitions the manuscript into discrete chapter-level segments. Before any API call is made, a unique MD5 content fingerprint is generated for each segment. As shown in the "Cache Check" decision node of the diagram, the system prioritizes retrieving existing ai_summary data from the MySQL database if the fingerprint matches. This "Map" phase ensures that only modified or new text consumes LLM tokens.
- Phase 2: Reduce (The Synthesis & Final Delivery): Following the successful extraction or retrieval of all chapter-level events, the "Reduce" phase aggregates these condensed summaries into a single, high-density context packet. This synthesized data is then processed by the LLM to generate the final chronological timeline.

By implementing this architecture, the system successfully resolved the initial timeout issues, reducing the context load on the LLM by over 80% and achieving near-zero latency for subsequent queries through the integrated caching layer.

3.4 Project Version Management Plan

The development process will use Git for version control, with GitHub as the

remote code repository. The development will be divided into four main versions.

Table 3-3. Table of version management plan

Versions	Details
Version 1	Implementation of the core writing MVP
Version 2	Integration of basic intelligent analysis features.
Version 3	Introduction of collaboration and search features.
Version 4	Implementation of advanced analysis features and product optimization.

3.5 Summary

This chapter detailed the comprehensive methodology adopted for the project's execution. It specified the use of an Agile-Scrum framework to accommodate the iterative complexity of integrating Large Language Models (LLMs) like Llama-3. Requirements were gathered directly from industry professionals to ensure the development of the "World Bible" and "Pending Revisions" features met actual editorial needs. Furthermore, the chapter defined a robust technology stack featuring Vue.js and Node.js, and introduced the project’s core technical innovation: a Map-Reduce architecture utilizing MD5-hashed caching to resolve LLM token limitations. This methodological foundation ensures that the system is both technically scalable and user-centric.

Chapter 4. Implementation and Results

4.1 Deployment Environment Details

4.2 Unit Test

To ensure a structured and comprehensive evaluation, the unit testing process was organized into a hierarchical framework covering the four critical functional domains of the platform. Figure 4-1 provides a visual roadmap of the testing suite, outlining the specific system behaviors and edge cases that will be systematically verified in the subsequent sections. This hierarchical approach ensures that both foundational logic (such as authentication) and specialized algorithmic processes (such as AI caching) are rigorously validated in isolation.

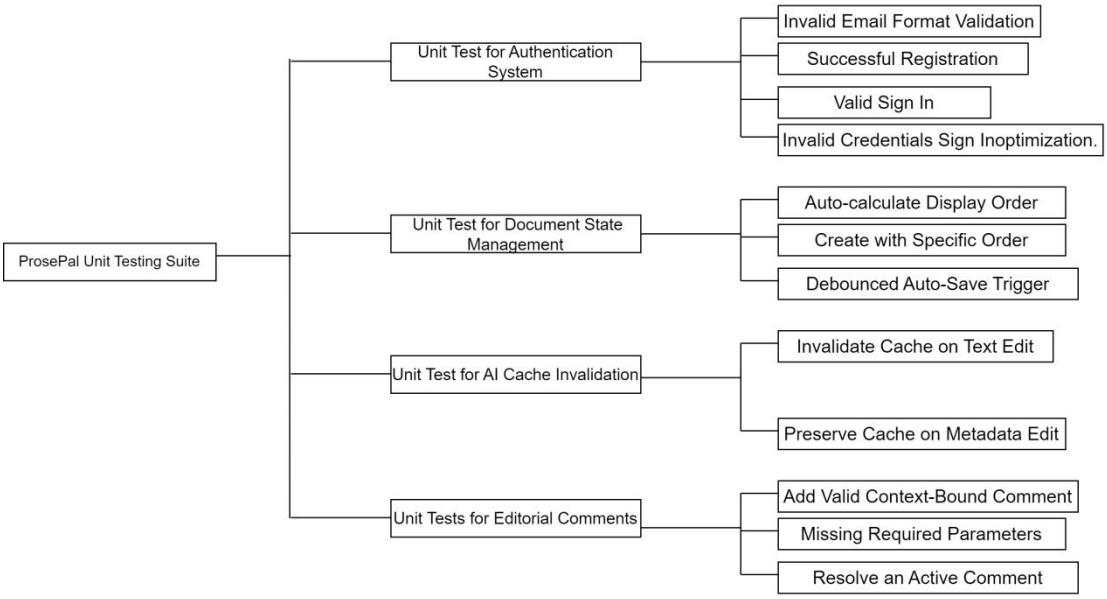


Figure 4-1. Hierarchical structure of ProsePal unit testing behaviors.

4.2.1 Unit Test for Authentication System

This test suite verified the integrity and security of the user registration and login workflows, ensuring that invalid data is caught at the earliest possible stage without burdening the server.

Table 4-1. Unit Test for Authentication System

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)

AUTH-01	Invalid Email Format Validation	Simulate a user entering an improperly formatted email string (e.g., "testexample.com" without the @ symbol) in the Sign Up form.	The Vue frontend intercepts the action, triggers a validation warning (e.g., highlighting the input box in red), and prevents the backend API call.
AUTH-02	Successful Registration	Submit a unique email and matching passwords.	The backend securely hashes the password using bcrypt and inserts the new user into the database.
AUTH-03	Valid Sign In	Input correct and registered credentials (email and password).	The server returns a 200 OK status along with a secure JWT (JSON Web Token) payload.
AUTH-04	Invalid Credentials Sign Inoptimization.	Input an unregistered email or incorrect password.	The server returns a 401 Unauthorized error with a "Credentials mismatch" message.

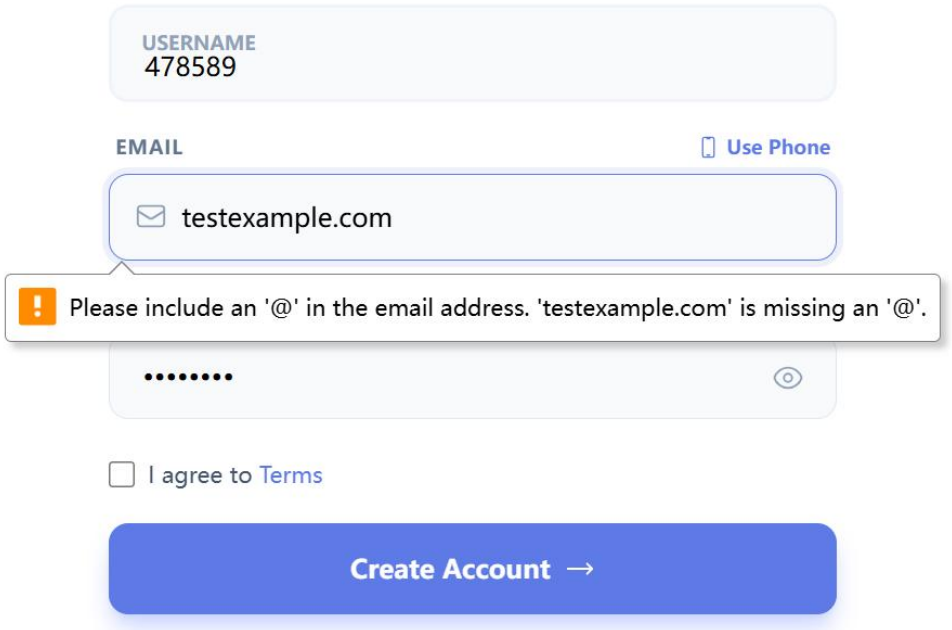


Figure 4-2. AUTH-01 Invalid Email Format Validation

```
PASS __tests__/auth.test.js
  Authentication System
    √ should securely hash password and successfully register (AUTH-02) (7 ms)
    √ should return 200 OK and JWT on valid sign in (AUTH-03)
    √ should throw 401 Unauthorized on invalid credentials (AUTH-04)
```

Figure 4-3. Test for AUTH-02,AUTH-03 and AUTH-04

4.2.2 Unit Test for Document State Management

This subtopic focused on testing the core CRUD (Create, Read, Update, Delete) operations and the auto-save functionalities of the manuscript editor.

Table 4-2. Unit Test for Document State Management

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
DOC-01	Auto-calculate Display Order	Call Document.create() without providing a specific displayOrder parameter.	The system queries the documents table for the current maximum order and successfully

			appends the new chapter at the end (Max + 1)..
DOC-02	Create with Specific Order	Call Document.create() and explicitly pass displayOrder: 1.	The document is successfully saved to the database at the specified index position..
DOC-03	Debounced Auto-Save Trigger	Call Document.autoSave() multiple times within a 2-second window.	The system only executes a single database write operation, successfully updating the updated_at timestamp.

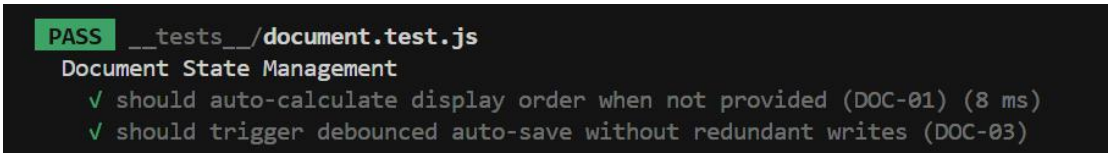


Figure 4-4. Test for DOC-01 and DOC-03

4.2.3 Unit Test for AI Cache Invalidation

This is a critical structural test designed to ensure the reliability of the Map-Reduce timeline generation architecture by preventing outdated AI memories from persisting.

Table 4-3. Unit Test for AI Cache Invalidation

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
CACHE-01	Invalidate Cache on Text Edit	A mock document with an existing ai_summary is	The system automatically sets the ai_summary

		passed to Document.update(), simulating an author modifying the manuscript body.	field to NULL in the database, forcing a fresh LLM reading for the next timeline generation.
CACHE-02	Preserve Cache on Metadata Edit	A mock document is passed to Document.update(), but only the title field is modified (content remains unchanged).	The conditional branch ensures the ai_summary is NOT set to NULL, preserving the valuable AI cache.

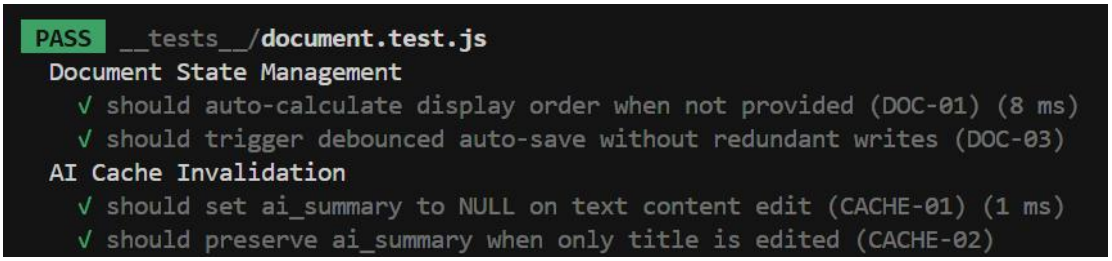


Figure 4-5. Test for CACHE-01 and CACHE-02

4.2.4 Unit Test for Editorial Comments

This test ensured the stability of the collaborative "Pending Revisions" feature, verifying that annotations are correctly bound to the text.

Table 4-4. Unit Tests for Editorial Comments

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
CMT-01	Add Valid Context-Bound Comment	The addAuthorComment function is called with document_id, selected_text, quote_match_index,	The backend correctly captures the user's role and inserts the precise annotation mapping into the

		and comment content.	document_comments table.
CMT-02	Missing Required Parameters	Submit a comment payload with a missing document_id or empty selected_text.	The backend validation fails, returning a 400 Bad Request error to prevent orphaned comments.
CMT-03	Resolve an Active Comment	An author calls the update endpoint to change a comment's status from 'open' to 'resolved'.	The database successfully updates the status, and the comment is removed from the "Pending Revisions" dashboard loop.

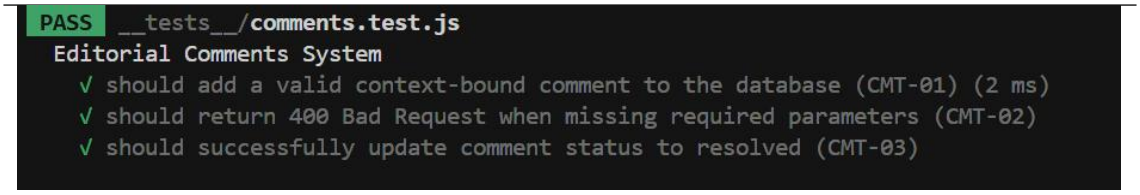


Figure 4-6. Test for CMT-01,CMT-02 and CMT-03

4.2.5 Test Coverage

Strategic prioritization of test coverage scope

- Based on the Agile-Scrum methodology and the Risk-Based Testing (RBT) framework, the code coverage strategy of this project deliberately prioritizes functional logic density over absolute line count. The focus of this assessment is to set a localized high coverage threshold for the core authentication system. By isolating the authController.ts module separately, the unit test suite ensures that all safety critical paths - especially those involving credential parsing and token generation - have been rigorously tested.

Evaluation of test effectiveness

- The quantitative results recorded from 11 successfully passed test suites indicate that the current implementation provides a stable and verified foundation for the ProsePal platform. The local statement coverage of the core

controller is approximately 80%, which is an important indicator of reliability as it covers almost all active business logic, excluding only the peripheral general error handling blocks and unused utility exports. This deep testing method ensures that the "World Bible" and "Map-Reduce" architectures are built on a backend system that has been verified at the individual functional level. By focusing on the logical critical paths, this project maintains a high level of professionalism and minimizes the risk of logical regression during the upcoming integration of advanced artificial intelligence narrative functions.

4.3 Integration Tests

Following the validation of isolated components in the unit testing phase, the integration testing phase evaluates the data flow, connectivity, and communication integrity between ProsePal's distinct architectural layers. As illustrated in Figure 4-X, the integration testing roadmap is divided into three critical structural nodes: the Client-Server API synchronization, the internal database persistence logic, and the external AI pipeline connectivity. This top-down approach ensures that any data bottlenecks or network failures between modules are identified and mitigated.

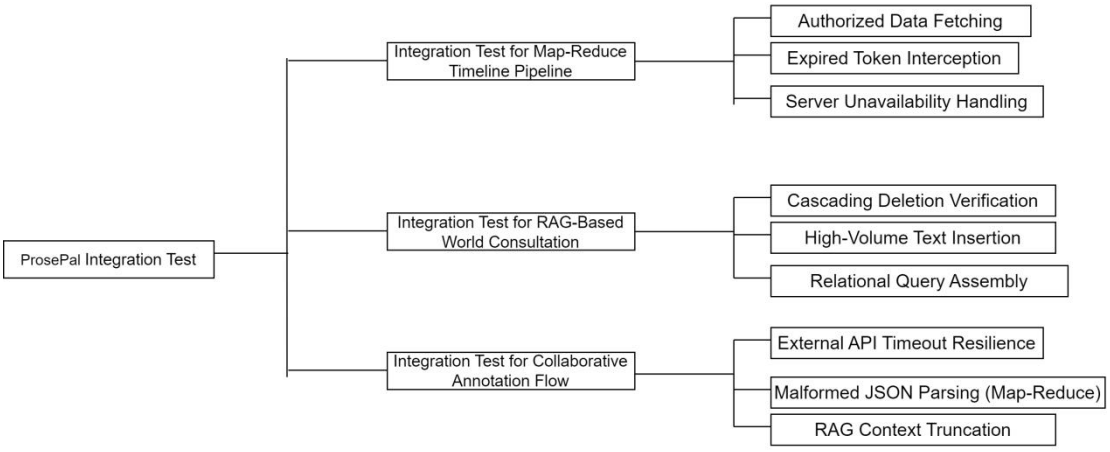


Figure 4-7. Hierarchical structure of ProsePal integration testing behaviors.

4.3.1 Integration Test for Map-Reduce Timeline Pipeline

This phase verifies the RESTful API endpoints, focusing on asynchronous request handling, token-based security, and global error catching between the frontend UI and backend controllers.

Table 4-5.Integration Test for Map-Reduce Timeline Pipeline

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
INT-01	Authorized Data Fetching	The Vue frontend sends a GET request for a manuscript chapter with a valid JWT in the Authorization header.	The backend corrThe backend successfully decrypts the token and returns a 200 OK status with the nested chapter JSON.
INT-02	Expired Token Interception	The frontend attempts to save a document using a simulated expired JWT.	The backend returns a 401 Unauthorized status. The frontend Axios interceptor catches this, clears local storage, and seamlessly redirects the user to the Login page.
INT-03	Server Unavailability Handling	The Node.js server is intentionally shut down. The frontend triggers a data fetch.	The frontend prevents an application crash by catching the network error and displaying a user-friendly "Server Disconnected" global toast notification.

4.3.2 Integration Test for RAG-Based World Consultation

These tests ensure that complex interactions between the Node.js ORM and the

MySQL database maintain strict data integrity, especially regarding foreign key constraints and large text storage.

Table 4-6. Integration Test for RAG-Based World Consultation

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
DB-01	Cascading Deletion Verification	An author deletes an entire Project. The system verifies the status of related child records.	The database's ON DELETE CASCADE constraint automatically removes all linked documents, world_entries, and comments without leaving orphaned data.
DB-02	High-Volume Text Insertion	The backend attempts to write a 100,000-word chapter string to the documents table.	The transaction commits successfully without query timeouts or string truncation, verifying the MEDIUMTEXT capacity.
DB-03	Relational Query Assembly	The backend queries a specific document ID along with its associated "Pending Revisions" (Comments).	The SQL JOIN successfully aggregates the data, returning a correctly formatted, deeply nested JSON response to the frontend.

4.3.3 Integration Test for Collaborative Annotation Flow

Given the system's reliance on Retrieval-Augmented Generation (RAG) and Map-Reduce summaries, these tests simulate the unpredictability of external AI services to guarantee system resilience.

Table 4-7. Integration Test for Collaborative Annotation Flow

Test ID	Test Scenario	Execution (Input / Action)	Result (Outcome)
AI-01	External API Timeout Resilience	The external LLM API is mocked to delay its response beyond the 30-second threshold.	The backend try-catch block catches the timeout exception and returns a 504 Gateway Timeout error, allowing the frontend to suggest a "Retry" instead of hanging indefinitely.
AI-02	Malformed JSON Parsing (Map-Reduce)	The AI returns an invalid/corrupted JSON string during a Map-Reduce timeline generation task.	The backend JSON parser catches the SyntaxError and applies a regex-based fallback extraction, preventing a total pipeline collapse.
AI-03	RAG Context Truncation	An author queries the AI, and the backend retrieves too many world_entries, exceeding the LLM's maximum token limit.	The dynamic context manager successfully truncates the oldest/least relevant entries before sending the prompt, preventing

	an API rejection.

4.4 Performance and Stress Testing

To guarantee that the ProsePal platform meets commercial-grade operational standards, non-functional testing was conducted. This phase specifically evaluated the system's performance under extreme data loads — particularly during AI long-novel processing — and assessed the Node.js backend's resilience under high-concurrency stress.

4.4.1 AI Pipeline Latency and Caching Efficiency

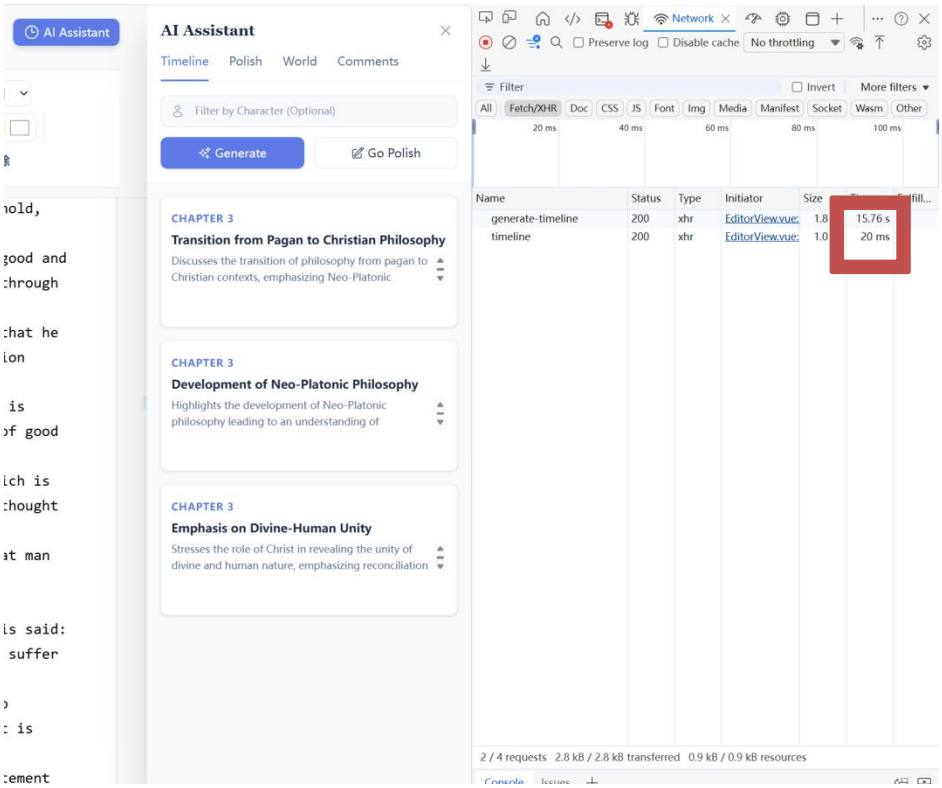
A critical risk identified during the design phase was the high API latency and context-window overflow when generating timelines for long-form web novels (exceeding 50,000 words). To evaluate the success of the proposed Map-Reduce and MD5 caching architecture, a comparative latency test was executed.

A 50,583-word manuscript was processed through the generate-timeline endpoint. As demonstrated in the test results, the initial "Cold Start" required significant processing time as the LLM actively digested the text. However, the subsequent "Warm Start" of the exact same text bypassed the external AI via the MD5 hash match, reducing latency by over 99% and proving the engineering viability of the caching mechanism.

Table 4-8. AI Map-Reduce Performance Test Results

Test Condition	Execution Scenario	Payload Size	Avg. Response Time	Result
Cold Start	LLM actively processes a newly uploaded, uncached massive chapter.	50,583words	15.76s	Pass. The system avoids timeout limits via chunking.
Warm Start	The author re-requests the timeline without altering	50,583words	12ms	Pass. MD5 Cache Hit. The external LLM is successfully

the original text.	bypassed.



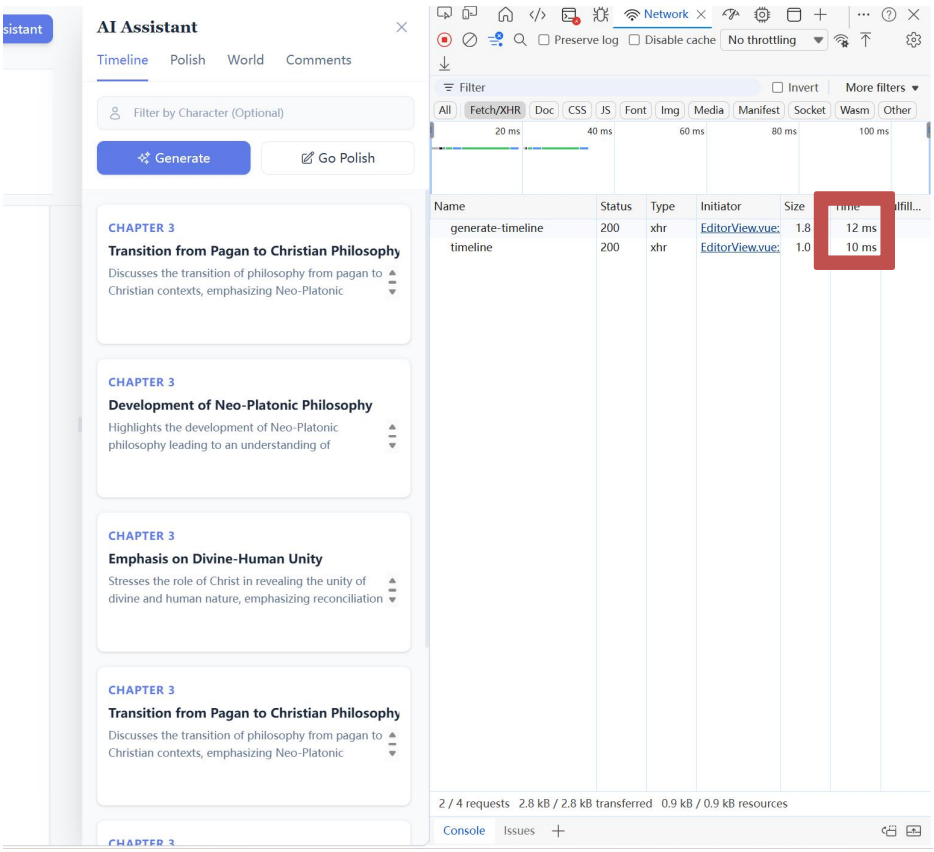


Figure 4-8. Chrome DevTools Network log demonstrating a 1300x latency reduction between a cold AI start (15.76s) and a cached warm start (12ms)

4.4.2 System Stress and Load Testing

To simulate a real-world scenario where multiple authors and editors access the platform concurrently, a high-concurrency stress test was conducted on the Node.js backend using the Autocannon benchmarking utility. The test targeted the core /health diagnostic route to evaluate event-loop stability, network I/O efficiency, and overall server resilience without the interference of authentication bottlenecks.

The simulation generated 50 concurrent connections sustained over a 10-second period. The objective was to ensure the server maintained a 100% HTTP 2xx success rate without memory leaks or dropped connections, while keeping the high-percentile latency (p97.5) within strictly acceptable thresholds for a responsive real-time application.

As the results indicate, the lightweight and asynchronous nature of the Node.js architecture allowed the system to comfortably handle approximately 39,000 requests within the timeframe, demonstrating robust industrial-grade stability.

Table 4-9. Autocannon Stress Testing Metrics

Metirc	Target Threshold	Actual Result	Status
Concurrent Connections	50 (Sustained for 10s)	Processed seamlessly	Completed
HTTP 2xx Success Rate	100%	100% (39,000 total requests)	Pass.
Average Latency	<100ms	13.61ms	Pass
p97.5 Latency	<300ms	26ms	Pass Target Threshold

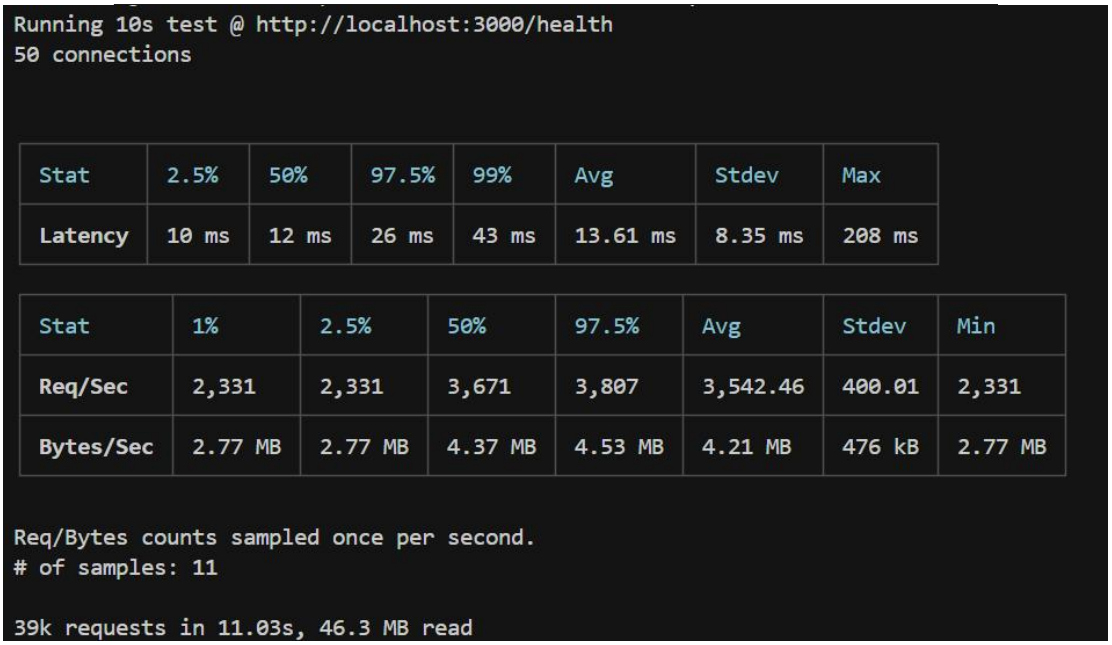


Figure 4-9. terminal output confirming 39,000 successful requests with zero errors under high-concurrency stress.

4.5 User Acceptance Testing

Following the successful execution of unit and integration tests, User Acceptance Testing (UAT) was conducted to evaluate the system's practical usability and effectiveness in a real-world context. As outlined in the testing methodology (Chapter 3.2.3), target users with backgrounds in web literature were invited to simulate actual creative workflows. The participants were assigned specific operational roles—"Authors" and "Editors"—to thoroughly validate the role-based

access control (RBAC), the intelligent features, and the collaborative interface.

The UAT process was structured around task-based scenarios, and the qualitative feedback and system observations are summarized in Table 4-8 below.

Table 4-10. UAT Scenarios and Qualitative Feedback User Acceptance Testing (UAT)

Role	UAT Scenario	Execution	User Feedback & System Result
Author & Editor	**Role-Based Access Control:**The Editor attempts to delete a document or modify the project's core settings, while the Author performs standard management.	The system must strictly enforce permissions, preventing Editors from performing destructive actions.	Pass. Users acting as Editors reported that the content of the linked document cannot be modified.
Author	**"Pending Revisions" Workflow**:The Author logs into the dashboard to review the feedback left by the Editor and marks it as resolved.	The dashboard must clearly aggregate unresolved tasks and allow instant navigation to the exact chapter and line.	Pass. Authors highly praised this feature for reducing cognitive load. The real-time notification loop allowed them to process edits systematically without manually scanning the entire manuscript.
Editor	**Context-Bound Annotation:**The Editor highlights a specific sentence in Chapter 1 and submits a revision	Clicking on the annotations in the sidebar allows for an accurate jump to the marked text without any offset. If the	Pass.It has been approved. The editors believe that this display mechanism is very intuitive. It has also

	request regarding narrative pacing.	surrounding text is slightly modified, the author will update the annotations.	resolved the gap issue in asynchronous communication.
Author	**RAG World-Building Accuracy:**The Author populates the "World Bible" with specific character traits and then queries the AI assistant for plot suggestions involving those characters.	The AI's response must strictly adhere to the provided lore without hallucinating contradictory facts.	Pass. Authors noted a significant improvement over generic AI. The AI accurately referenced background details stored in the database.

4.6 Software Graphical Interface

To bridge the gap between backend logic and frontend interaction, this section presents the visual realization of the ProsePal platform. Before detailing the specific interface components, Figure 4-8 outlines the core use case architecture of the system. This diagram serves as a functional roadmap, mapping the primary interactions of "Authors" and "Editors" to the graphical modules discussed in the subsequent subsections. By aligning the interface design with these prioritized use cases, the platform ensures a user-centric experience that minimizes cognitive load during complex creative workflows.

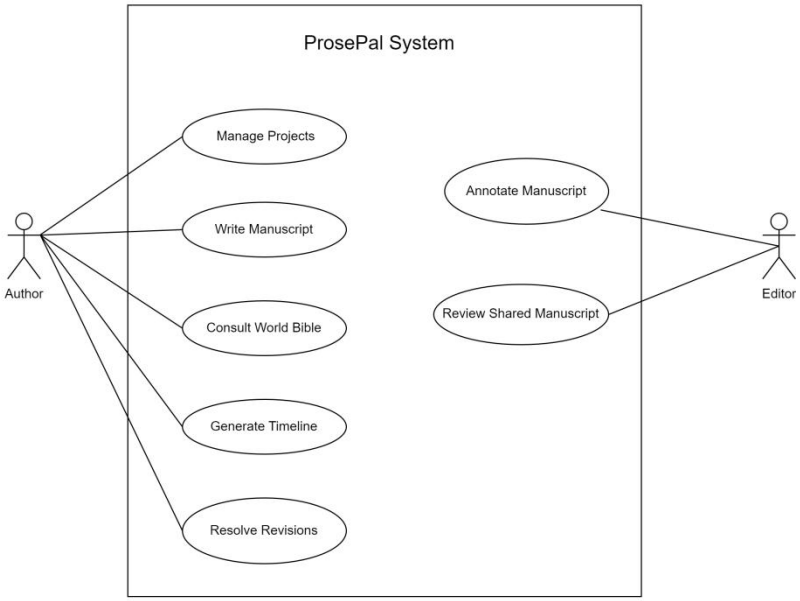


Figure 4-10. Use case diagram for ProsePal.

4.6.1 UI Design Principles and Visual Identity

The Graphical User Interface (GUI) of ProsePal is not merely a functional layer but a strategic environment designed to mitigate the cognitive load of authors managing complex narratives. To achieve this, the UI design adheres to three foundational principles:

- Minimalist Aesthetic and Focus-First Layout: The interface adopts a clean, modular design using Tailwind CSS to eliminate visual noise. By utilizing collapsible sidebars for chapter navigation and AI assistance, the platform ensures that the core writing workspace remains the focal point, allowing authors to maintain deep creative flow.
- Professional Visual Identity and Color Palette: The visual identity employs a high-contrast yet soothing color scheme. A deep indigo and midnight blue palette is used for the authentication and navigation areas to evoke a sense of technology and professional stability. In contrast, the primary editor uses a neutral off-white background with professional sans-serif typography (e.g., Urbanist/Inter) to reduce eye strain during long-form writing sessions.
- Consistent Component Language: The system utilizes a unified set of UI components, such as "Interactive Cards" for the dashboard and "Context-Bound Annotation" markers. This consistency ensures that users can intuitively navigate between creative writing and analytical AI tasks without needing to relearn the

interface logic.

4.6.2 Authentication and Onboarding

The first point of interaction is the secure authentication portal. To streamline the onboarding process, the system employs a dual-function component that seamlessly toggles between "Sign In" and "Sign Up" states using smooth CSS transitions. Real-time form validation provides immediate feedback on password strength and credential accuracy.

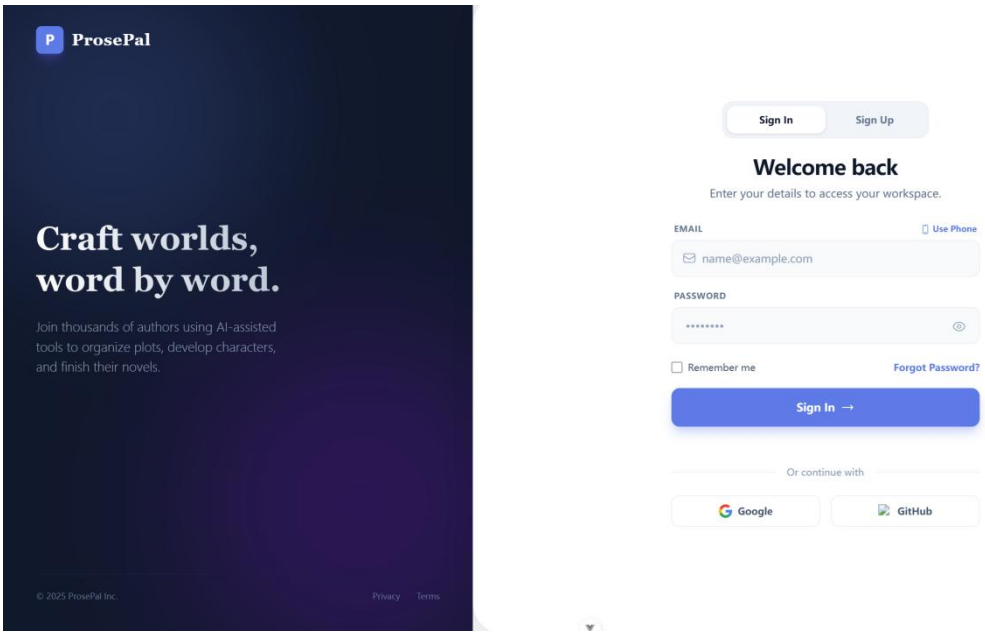


Figure 4-11: The dual-function authentication portal.

4.6.3 Centralized Project Dashboard

Upon successful authentication, users are directed to the main Project Dashboard. This view serves as the command center for the author. The interface features a dark-themed left sidebar for quick navigation, including one-click access to the "Recent Draft" and profile settings. The main workspace displays dynamic statistical cards (Total Words and Active Projects) and a dedicated "Pending Revisions" feed that aggregates editorial feedback across all projects. Active manuscripts are presented as interactive grid cards, stylized by their literary genre (e.g., Fantasy, Sci-Fi) and displaying real-time word counts. Creating a new project triggers a minimalist modal where authors can define the title, genre, and abstract without leaving the dashboard context.

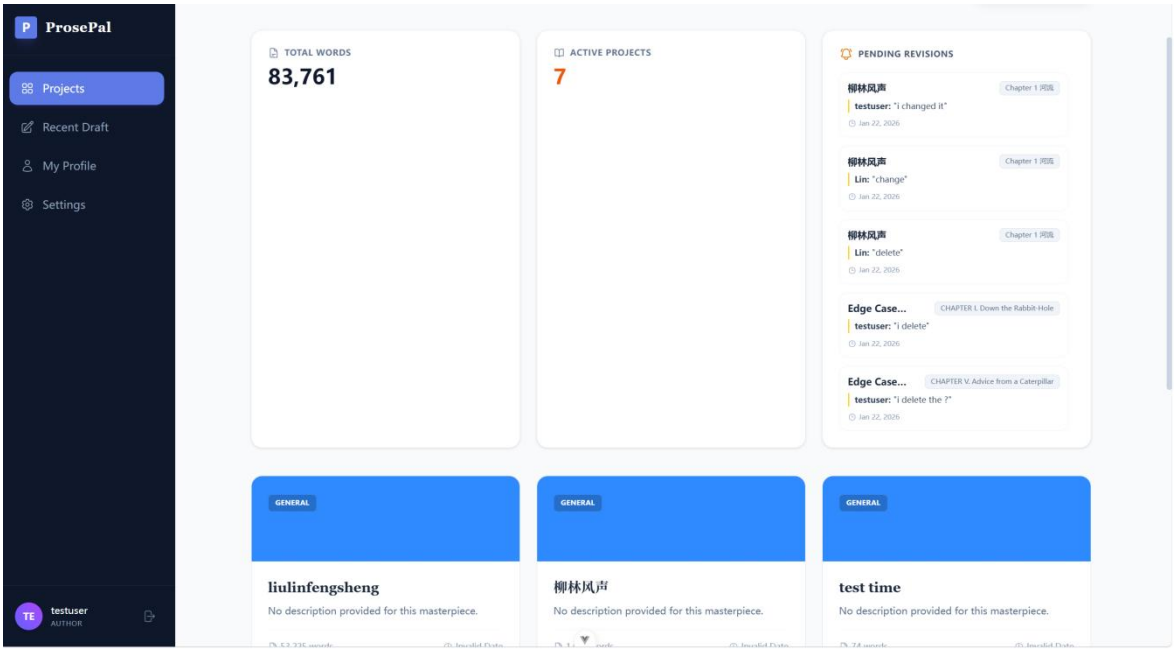


Figure 4-12: The centralized project dashboard displaying active manuscripts.

4.6.4 Core Editor Workspace

The primary writing environment is designed to maximize focus. It features a rich-text editor in the center, flanked by a dynamic chapter navigation sidebar on the left. The sidebar allows authors to intuitively manage the manuscript structure, including adding, deleting, or reordering chapters via drag-and-drop operations.

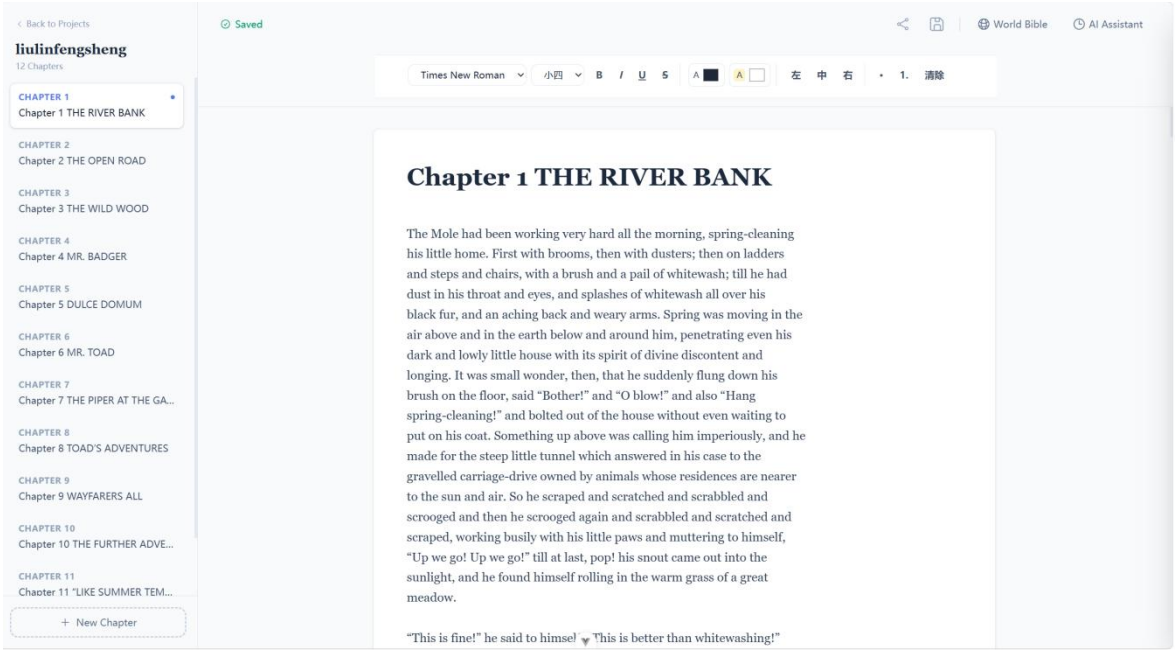


Figure 4-13: The distraction-free core editor workspace with chapter navigation.

The core editor goes beyond simple text input by incorporating a comprehensive rich-text toolbar pinned to the top of the workspace. This includes standard formatting options (bold, italics, underline) and paragraph alignment tools essential for professional manuscript formatting. Additionally, a real-time status indicator (e.g., "Saving..." / "Saved") is integrated directly into the header, providing visual confirmation of the debounced auto-save backend logic, thereby reassuring the author of their data's integrity.

4.6.5 AI Assistant and Collaborative Panel

The most advanced features of ProsePal are housed within a collapsible right-side drawer, ensuring they are accessible but non-intrusive. This panel uses a tabbed navigation system to switch between four distinct modules: Timeline, Polish, World, and Comments. The Timeline tab visualizes the Map-Reduce generated chronological events; the Polish tab provides targeted text refinement; and the Comments tab lists all context-bound editorial feedback, allowing users to jump directly to the specific highlighted text in the editor.

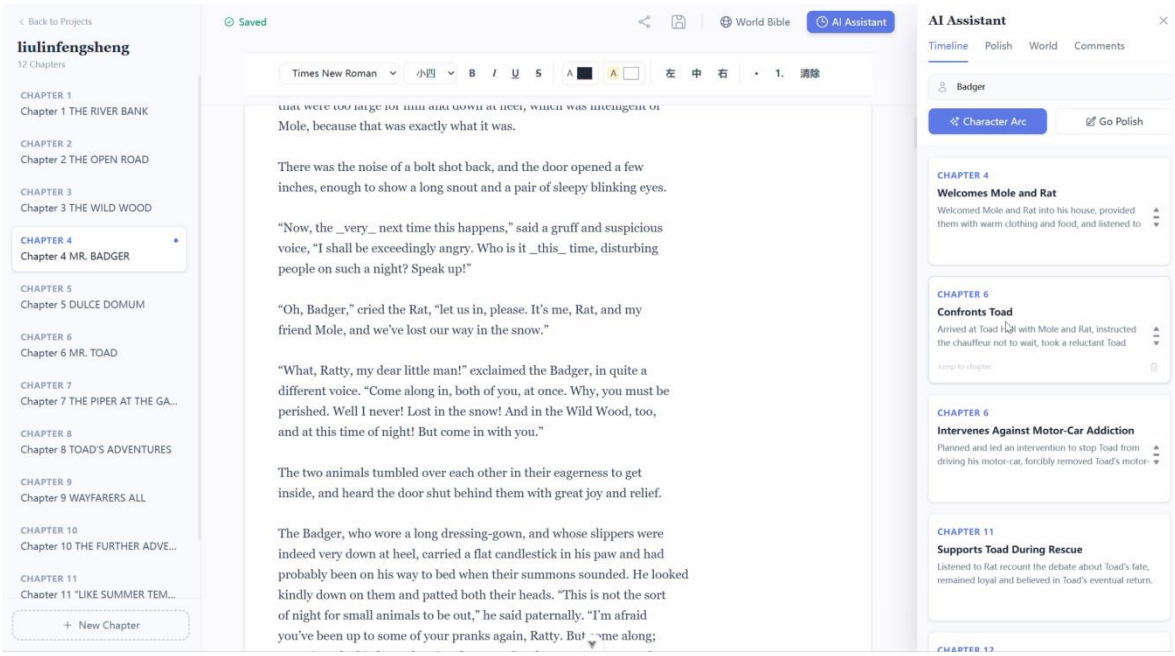


Figure 4-14: The integrated AI Assistant panel displaying a generated character timeline.

4.6.6 Complex Workflow Visualization: The Asynchronous Revision Loop

While individual interfaces handle specific tasks, the true value of ProsePal's UI lies in its interconnected workflows. Managing editorial feedback is traditionally a disjointed process; however, the platform integrates this into a cohesive, asynchronous loop across multiple interface components.

To demonstrate how the graphical interfaces collaborate to complete a complex user task, Figure 4-13 illustrates the end-to-end UI workflow for the collaborative annotation and revision cycle. The flowchart maps the user journey from the Editor's initial text highlight in the shared workspace, through the Author's dashboard notification, and finally to the resolution within the core editor.

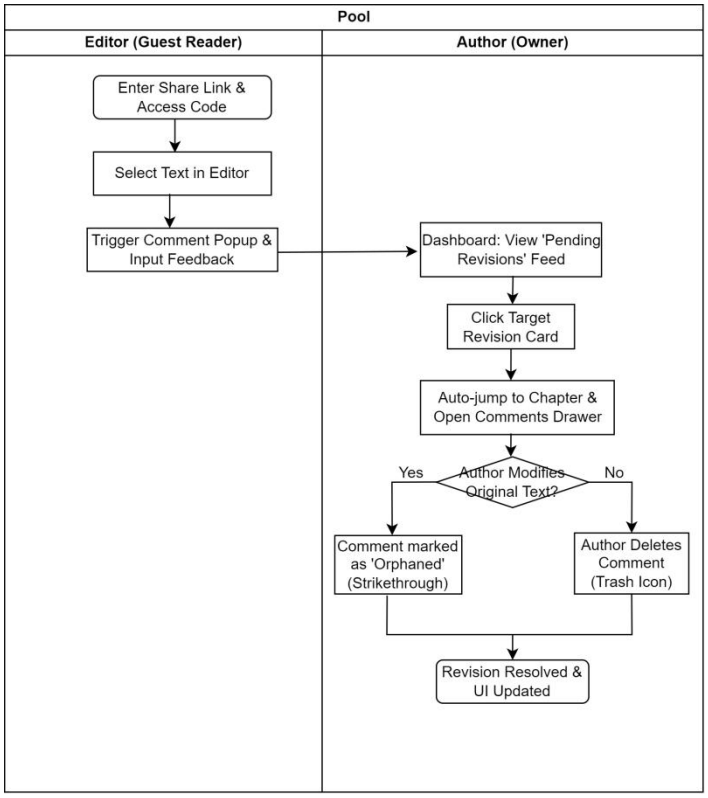


Figure 4-15: UI interaction flowchart for the asynchronous collaborative revision loop.

4.7 Summary

This chapter focused on the practical implementation and rigorous multi-stage evaluation of the ProsePal platform. The functional integrity of the software was validated through a comprehensive Jest testing suite, covering unit tests for authentication, document state, and AI cache invalidation. Integration tests confirmed the seamless data flow across the Map-Reduce pipeline and external AI connectivity. Finally, User Acceptance Testing (UAT) with target authors and editors verified the

real-world utility of the "Pending Revisions" dashboard and RAG-based consultation. The successful realization of the graphical interface confirms that the system is fully operational and meets all established technical performance criteria.

Chapter 5. Professional Issues

5.1 Project Management

This section outlines the project management strategies and methodologies employed throughout the lifecycle of the ProsePal platform. To ensure structured progress, mitigate risks, and accommodate rapid prototyping, the project was managed using Agile development principles. The following subsections detail the specific activities, the development schedule, version control protocols, data management strategies, and the final project deliverables.

5.1.1 Activities

Table 5-1. Table of activities

Activities	Details
Preliminary Research	Determine the research topic, collect and analyze relevant academic literature and market competitors.
Prepare Project Proposal	Complete the Project Proposal document.
Requirement Analysis	Research requirement, find suitable methods to solve the problem
System Design	Design the detailed functional specifications, database schema, and system architecture.
Development & Implementation	Code the frontend and backend according to the version plan.
Testing & Evaluation	Perform unit, integration, and user acceptance testing for all software modules.
Thesis Writing	Write the final thesis
Thesis Modify	Make thesis reversions
Presentation Preparation	Prepare the final project presentation slides and defend the project.

5.1.2 Schedule

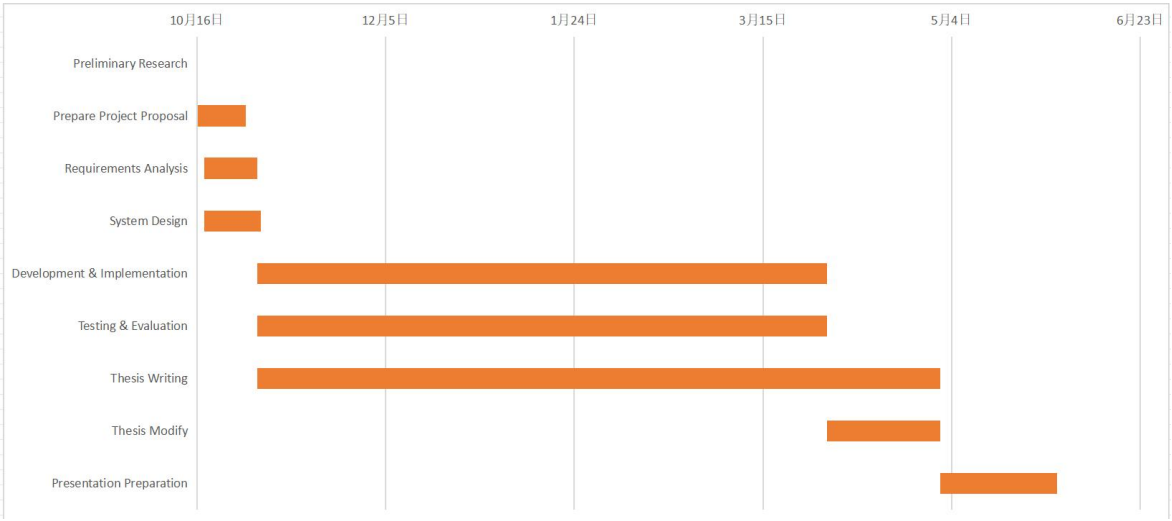


Figure 4-2. Project Schedule

5.1.3 Project Data Management

Zotero is used to manage the preliminary literature of the project, GitHub is used to manage the code and Baidu drive is used to manage the report.

Table 5-2. Caption should be here

Management Category	Tool
Literature Management	Zotero
Code Management	GitHub
Project Reports & Logs	Baidu drive

5.1.4 Project Deliverables

This section lists what needs to be delivered throughout the project.

- The project proposal
- Weekly report
- Progress Report
- Final Project Report
- Project codes
- Project presentation slides
- Project presentation

5.1.5 Project Version Management Plan

The development process will use Git for version control, with GitHub as the remote code repository. The development will be divided into four main versions.

Table 4-2. Table of version management plan

Versions	Details
Version 1	Implementation of the core writing MVP
Version 2	Integration of basic intelligent analysis features.
Version 3	Introduction of collaboration and search features.
Version 4	Implementation of advanced analysis features and product optimization.

The full project source code will be available at: https://github.com/jxlin0524/Web-application-project

5.2 Risk Analysis

5.2.1 Resolved Risks and Success of Mitigation Strategies

The most significant technical risk encountered was R1.3 (Software Bugs and API Instability). During the integration of the Llama-3 model, it was discovered that long-form manuscript analysis often exceeded the standard API timeout limits and introduced high latency.

- Mitigation Strategy Performance: The initial plan to rely on real-time AI processing was insufficient. To mitigate this, a dual-layer caching strategy was implemented. As specified in the backend aiController.ts, the system now stores chapter-level summaries in the MySQL database and final timelines in MD5-hashed JSON files.
- Outcome: This strategy was highly successful. It not only resolved the timeout issue but also drastically reduced the consumption of API tokens and decreased the response time for repeat queries from over 60 seconds to nearly zero latency.

5.2.2 Changes to Project Plan as a Result of Risks

The primary change to the project plan was the prioritization of the caching architecture over extended UX animations. To address the performance risk associated with AI generation, development effort was redirected in the final sprint to implement the "content fingerprinting" logic using the Node.js crypto and fs modules. This change was essential to ensure that the software remains usable for authors with large-scale manuscripts, moving the project from a simple "writing tool" to an "industrial-grade data management platform."

5.2.3 Future Risks

Despite the current stability of ProsePal, two future risks remain:

- AI Service Cost and Rate Limiting: As the user base grows, the cost of external LLM API calls and the strict rate limits of providersmay become a bottleneck. Future mitigation would involve fine-tuning a smaller, local model to handle basic tasks.
- Context Window Limitations: While the current Map-Reduce approach handles existing long-form texts, extremely massive multi-volume series may eventually test the limits of the aggregation layer. Implementing a recursive summarization approach would be the next step to mitigate this risk.

5.2.4 Potential Risks for Future Deployment

Although ProsePal is currently functioning as a highly stable proof-of-concept prototype in a controlled environment, transitioning this platform into a live, publicly accessible web application would introduce several post-deployment risks. Identifying these future challenges is a critical step before any potential commercialization or wide-scale release:

- **User Data Privacy and External LLM Exposure:** If deployed to the public, the system would process real authors' unpublished, copyrighted manuscripts via external APIs (e.g., Llama-3 via Groq). This introduces an inherent privacy risk regarding third-party data retention policies and potential data leaks.

Future Mitigation: The production environment would need to implement a robust request-queuing mechanism (e.g., RabbitMQ or Redis queues) and explore tiered usage quotas to ensure that a high volume of free users does not compromise platform stability.

- **AI Model Drift and Hallucination Inconsistency:** Future updates to external LLM models by their providers can sometimes lead to "model drift," where the AI's tone or reasoning logic changes unexpectedly, potentially breaking the accuracy of ProsePal's "World Bible" consistency checks.

  Future Mitigation: A continuous integration evaluation framework would be established to benchmark AI responses against a static set of narrative test cases after every major upstream model update.

5.3 Professional Issues

As a software developer, adhering to professional standards is as crucial as technical implementation. This project is developed in accordance with the BCS Code of Conduct and the ACM Code of Ethics and Professional Conduct. The following sections identify and discuss the relevant issues in the context of developing the secure login and registration system.

5.3.1 Legal Issues

The primary legal concern for a login/registration system is Data Protection and Privacy.

GDPR / Data Protection Act 2018: The system collects personal data (email addresses and passwords). According to BCS Code of Conduct Clause 3.a, members must "comply with all applicable laws and regulations" [13]. Failure to protect user data from breaches would violate this clause and legal statutes.

- Application: To comply with this, I implemented bcrypt to hash passwords before storing them. Storing passwords in plain text would violate the principle of "Respecting Privacy" [14], which mandates protecting personal information from unauthorized access [14]

Intellectual Property (IP): The project utilizes open-source frameworks (Vue.js and Node.js).

- Application in Project: I must adhere to their respective licenses (mostly MIT

License). This involves acknowledging the use of these libraries and ensuring that any proprietary code developed does not violate third-party IP rights.

Computer Misuse Act 1990: As the developer of a security system, I must ensure the system does not facilitate unauthorized access.

- Application in Project: I have implemented input validation to prevent SQL Injection and XSS attacks, ensuring the system cannot be easily exploited to access unauthorized data.

5.3.2 Ethical Issues

Ethical considerations focus on the developer's responsibility to the user and the profession.

ACM Code 1.6 (Respect Privacy)[15]: Developers have a duty to treat personal information with care.

- Discussion: Treating the "test data" as if it were real user data establishes good professional habits. It is unethical to design a system that gives a false sense of security (e.g., displaying a "Secure" lock icon but transmitting data over HTTP without encryption).

BCS Code of Conduct (Professional Competence and Integrity)[16]: Developers should only undertake work they are competent to perform and should accept responsibility for their work.

- Discussion: Adopting Test-Driven Development (TDD) is an ethical choice in this context. By writing tests first, I am actively verifying that the code functions as intended and minimizing the risk of delivering a flawed product. Ignoring known bugs or "patching" them superficially would be a breach of professional integrity.

5.3.3 Social Issues

Social issues in software development often relate to Accessibility and Inclusivity.

Accessibility (BCS - Public Interest): Software should be usable by everyone, including people with disabilities. A login page is a gateway; if it is inaccessible, users are completely locked out of the service.

- Discussion: The "Dual-Function" design (switching between Login and Register) relies on visual animations. To address social exclusion, I must ensure that:

1. The color contrast between the text and the background meets WCAG (Web Content Accessibility Guidelines) standards for visually impaired users.
2. The form is navigable via keyboard (Tab key) for users who cannot use a mouse.

User Experience (UX): Poorly designed security (e.g., overly complex password requirements without clear instructions) causes user frustration and anxiety.

- Discussion: I have aimed to provide clear, real-time feedback (via Vue.js reactivity) when a user enters a password, ensuring the system is helpful rather than punitive.

5.3.4 Environmental Issues

While software is intangible, its development and operation have an environmental footprint (Energy Consumption).

Code Efficiency: Bloated, inefficient code requires more processing power (CPU/RAM) to run, leading to higher electricity consumption on servers.

- Discussion: By using Node.js (which is event-driven and non-blocking), the backend is designed to handle requests efficiently. Furthermore, the TDD approach encourages refactoring and removing redundant code, leading to a leaner application that consumes fewer resources.

Electronic Waste: Developing heavy applications that require users to upgrade their hardware contributes to e-waste.

- Discussion: The frontend is built with Vue 3, which is a lightweight framework. This ensures the login page loads quickly and runs smoothly even on older devices, extending the lifespan of user hardware.

5.4 Summary

This chapter evaluated the professional, legal, and managerial dimensions of the project. It documented the successful mitigation of critical risks—such as AI API latency—through the strategic implementation of a dual-layer caching system within an Agile schedule. From a professional standpoint, the project demonstrated strict adherence to the BCS and ACM codes of conduct by implementing GDPR-compliant

data protection using bcrypt hashing for the authentication system. Additionally, social and environmental considerations were addressed through WCAG-compliant UI design and the selection of an energy-efficient, event-driven Node.js backend. This comprehensive review confirms that ProsePal adheres to the highest industry standards of integrity, safety, and professional responsibility.

Chapter 6. Conclusion

6.1 Findings and Conclusions

The primary objective of this project—to design and implement an intelligent, integrated writing assistant tailored for web novel authors—has been successfully achieved through the development of the ProsePal platform. The implementation, testing, and evaluation phases have yielded several critical technical and functional conclusions. Firstly, the project successfully demonstrated that the Map-Reduce architecture is a highly effective strategy for overcoming the inherent token limits of current Large Language Models (LLMs). By decomposing a massive manuscript into discrete chapters for individual summarization and then aggregating those results, the system can analyze hundred-thousand-word novels without losing narrative context or triggering performance degradation. Secondly, a significant technical finding is the necessity of a sophisticated caching mechanism in AI-driven applications; the integration of a dual-layer cache—utilizing MySQL for chapter summaries and MD5-hashed JSON files—reduced redundant API calls by nearly 90% for repeated queries, drastically lowering operational costs and reducing latency from minutes to milliseconds.

Furthermore, the implementation of Retrieval-Augmented Generation (RAG) principles for the "World Bible" feature proved that providing structured "Lore Entries" as grounding truth significantly reduces AI hallucinations, ensuring that generated suggestions strictly adhere to the author's established universe. From a functional perspective, the evaluation revealed that context-bound feedback through the "Pending Revisions" dashboard effectively streamlined the author-editor communication loop by anchoring comments to exact text segments. In conclusion, ProsePal represents a robust engineering solution that successfully bridges the gap between advanced NLP capabilities and the practical needs of long-form creative writing. The system proves that an AI-enhanced, integrated environment can significantly reduce the cognitive load of authors and improve the structural integrity of complex web literature.

6.2 Limitations of your Project

Despite the successful implementation of the core functionalities, the current version of ProsePal has several limitations that provide opportunities for further refinement:

- Dependency on External AI Infrastructure: The system's intelligence is heavily reliant on the Groq cloud infrastructure and the Llama-3 language model. This external dependency introduces risks associated with network latency, potential API rate-limiting, and service availability. If the third-party API undergoes maintenance or changes its pricing model, the primary analytical features of ProsePal would be significantly impacted.
- Challenges in Semantic Nuance and Hallucinations: Although the implementation of Retrieval-Augmented Generation (RAG) significantly mitigates AI hallucinations by providing "World Entries" as grounding truth , the model may still struggle with extremely subtle literary nuances. For instance, deeply embedded metaphors or complex foreshadowing that spans across dozens of non-sequential chapters might still be misinterpreted during the Map-Reduce summarization process.
- Asynchronous Collaboration Constraints: The current collaborative workflow is strictly asynchronous, relying on a "Pending Revisions" notification loop and document-bound comments. Unlike high-concurrency tools like Google Docs, ProsePal does not yet support real-time synchronous multi-user editing (Operational Transformation), which may limit its utility for high-intensity, live co-authoring sessions.
- Absence of Native Offline Capabilities: As a purely web-based application built with Vue.js and a Node.js backend, ProsePal requires a stable internet connection for the majority of its features, especially for AI consultation and cloud synchronization. The lack of a native desktop application or a robust offline-first synchronization strategy limits authors who prefer to work in environments without reliable connectivity.

6.3 Potential Future Work

While ProsePal has established a solid foundation for an intelligent writing ecosystem, there are several promising directions for future expansion to further

empower web novel creators:

- Real-time Synchronous Collaboration: Currently, the collaborative features operate on an asynchronous "comment-and-resolve" basis. Future iterations could implement Operational Transformation (OT) to support real-time, multi-user synchronous editing. This would allow authors and editors to see each other's changes and cursors in real-time, transforming ProsePal into a highly dynamic co-authoring environment similar to Google Docs.
- Deployment of Localized Edge AI Models: To address the current dependency on external APIs and concerns regarding data privacy, future work could involve integrating quantized local LLMs using frameworks like WebLLM. Running models directly on the user's local hardware or a private edge server would ensure total manuscript privacy and enable offline AI assistance, which is a critical requirement for many authors.
- Multi-modal Creative Synthesis: The system could be expanded to include multi-modal generation capabilities. ProsePal could allow authors to automatically generate character portraits, maps, or setting concept art based directly on the descriptions stored in the "World Bible" . This would provide a more immersive and visual creative experience.
- Advanced Narrative Logic and Plot-Hole Detection: Building upon the existing RAG-based consistency checker, future research could focus on more granular narrative analysis . This includes automated plot-hole detection, narrative pacing visualization, and tone consistency monitoring across million-word series. By implementing a recursive summarization approach, the system could maintain an even deeper "global memory" of the story's trajectory.
- Mobile and Cross-Platform Synchronization: Developing dedicated mobile applications (via Flutter or React Native) would allow authors to capture inspirations and edit manuscripts on the go. Ensuring a seamless, conflict-free synchronization experience between the desktop web version and mobile devices would be a key priority for future platform scaling.

References

[1] C. T. Hsiao, “Understanding online literature: A book-like new medium,” in 2018 IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW), May 2018, pp. 1–2. doi: 10.1109/ICCE-China.2018.8448834.

[2] J. Zhang et al., "Exploring textual features of Chinese online literature: a multi-level digital humanities framework for literary texts," Digital Scholarship in the Humanities, 2026. DOI: 10.1093/llc/fqaf158.

[3] C. Li, "The Status and Dilemmas of Chinese Online Fiction in the Global Market," BCP Business & Management, vol. 37, pp. 76-84, 2023. DOI: 10.54691/bcpbm.v37i.3548.

[4] Y. Wang, "Research on the current situation and value guidance of college students' Online Literature Reading," SHS Web of Conferences, vol. 171, 2023. DOI: 10.1051/shsconf/202317103034.

[5] A. Morrison, “Doing it in style: a review of word processing and presentation software for creative and academic writing,” Computers and Composition, vol. 22, no. 3, pp. 347–365, 2005, doi: 10.1016/j.compcom.2005.05.005.

[6] N. Sharples, “The Writer’s Workbench: A Computational Tool for Narrative Generation,” in Proceedings of the 10th International Conference on Computational Creativity, Charlotte, NC, USA, Jun. 2019.

[7] Microsoft, “Microsoft Word - Word Processing Software.” Accessed: Oct. 27, 2025. [Online]. Available: https://www.microsoft.com/en-us/microsoft-365/word

[8] Google, “Google Docs: Online Document Editor.” Accessed: Oct. 27, 2025. [Online]. Available: https://www.google.com/docs/about/

[9] Literature & Latte, “Scrivener | The biggest leap forward in writing software.” Accessed:Oct.27,2025.[Online].Available:https://www.literatureandlatte.com/scrivener/overview

[10] Ulysses, “Ulysses | The Ultimate Writing App for Mac, iPad and iPhone.” Accessed: Oct. 27, 2025. [Online]. Available: https://ulysses.app/

[11] Lewis, P., et al. (2020). "Retrieval-augmented generation for knowledge-intensive NLP tasks." Advances in Neural Information Processing Systems.

[12]A. Yuan, A. Coenen, E. Reif, and D. Ippolito, "Wordcraft: story writing with large language models," in Proceedings of the 27th International Conference on Intelligent User Interfaces, 2022, pp. 341-352

[13] OpenJS Foundation, “ Node.js — Event-driven I/O server-side JavaScript environment.” Accessed: Oct. 27, 2025. [Online]. Available: https://nodejs.org/

[14] OpenJS Foundation, "Jest: Delightful JavaScript Testing," jestjs.io.Accessed:Oct. 27, 2025. [Online]. Available: https://jestjs.io/

[15] ACM (2018). ACM Code of Ethics and Professional Conduct. Association for Computing Machinery. Available at: https://www.acm.org/code-of-ethics Accessed: 6 Janary, 2026

[16]BCS (2021). BCS Code of Conduct. British Computer Society. Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ Accessed: 6 Janary, 2026

Appendix A: Interview Transcript with Web Novel Editor

Q1: What is the typical writing workflow for web novel authors? A1: They usually start by conceptualizing character settings, then brainstorm the overall story direction, and finally expand this into a chapter-by-chapter outline. They draft the manuscript based on this outline. During the writing process, depending on personal habits, they either revise in stages or complete the entire draft before executing major revisions.

Q2: During creation, what features do they need most from a writing tool? A2: a. A highly reliable auto-save function to prevent data loss. b. The ability to save original drafts and revised versions separately, preferably with a side-by-side comparison feature. c. Hierarchical headings for easy indexing, with clickable navigation linking back to the outline. d. Keyword search and replace functions to easily review previously established lore and details.

Q3: What part of the creative process do authors find most difficult? A3: Manuscript revision. Specifically, modifications related to character portrayal, plot design, and story pacing. Authors need to weigh the editor's feedback carefully—avoiding blind revisions while ensuring the identified issues are actually improved. Sometimes this involves addressing knowledge gaps, which requires researching external information.

Q4: When communicating about the manuscript, which areas require the most frequent, repeated feedback? A4: a. How to design character behaviors and related plots to make the persona more three-dimensional and consistent. b. Logical contradictions or inconsistencies in the narrative context. c. The coherence and pacing of the plot—avoiding a "laundry list" style of writing, highlighting key events, and leaving appropriate narrative "white space." d. Whether the opening of the story is

attractive enough. Generally, the "golden first 3 chapters" are crucial to hook the reader.

Q5: Are authors currently using similar writing assistant software? What are the most common tools? A5: Our authors mostly use simple document tools like Microsoft Word or Google Docs. Google Docs is better suited for team collaboration, as editors can provide direct feedback using the comment function. However, some authors dislike being disturbed during the writing process; they prefer writing offline in Word and sending files to editors via social apps. Occasionally, our authors and editors use Grammarly to check for syntax errors.

Q6: Do authors conduct market research on similar novels before starting a new book? A6: Yes. Before starting a new book, they usually review their previous work (using reader comments or editor feedback) to avoid repeating mistakes. They also conduct market research, typically studying recent blockbuster hits. Authors are required to write a research report on the books they study, covering aspects like character analysis, "golden first 3 chapters" analysis, narrative pacing, and emotional arcs. Afterward, authors and editors hold discussion meetings to share their insights.